
QUOTAMESH

MVP Architecture and Product Specification v0.3

Local AI capacity wallet and project-safe failover manager

Status: Implementation source of truth for the MVP

Date: 2 October 2026

Primary builder: Solo developer with AI-assisted coding

Deployment: Local machine only

Architecture bias: Small, deterministic, testable, zero external infrastructure

Final product sentence

Connect the free, trial and paid AI API access you already own once; allocate it to project profiles; see what each project can safely use right now; and fail over automatically without unexpected spend.

Contents

1. Competitive reality in October 2026

- 1.1 What the current market already does well
- 1.2 The real gap QuotaMesh should target
- 1.3 What not to market as unique
- 1.4 What to market instead

2. Target user and scenarios

- 2.1 Primary user
- 2.2 Canonical mixed-capacity scenario
- 2.3 Failure scenario
- 2.4 Important compliance boundary

3. MVP product principles

4. Final MVP scope

- 4.1 Ship in v0.1
- 4.2 Stretch only if the core is complete
- 4.3 Deferred to v0.2+
- 4.4 Explicitly reject from MVP
- 4.5 Feature decision matrix from the pasted proposal

5. User experience and information architecture

- 5.1 First-run flow
- 5.2 Screen 1 - Capacity Wallet (home)
- 5.3 Screen 2 - Project Profiles
- 5.4 Screen 3 - Available Now
- 5.5 Screen 4 - Keys
- 5.6 Screen 5 - Activity / History
- 5.7 Screen 6 - Doctor
- 5.8 Screen 7 - Access Catalog

6. System architecture

- 6.1 Process model
- 6.2 Technology choices
- 6.3 Recommended source layout
- 6.4 Functional core, imperative shell

7. Request lifecycle and routing engine

- 7.1 Gateway flow
- 7.2 Raw pass-through rule
- 7.3 Core objects
- 7.4 Selection algorithm

7.5 Attempt limits

7.6 When all candidates are blocked

8. Health and quota state model

8.1 Persisted state scopes

8.2 Unknown 429 cooldown ladder

8.3 Explicit information wins

9. Failure classification matrix

10. Streaming rules

10.1 Commit point

10.2 Usage capture

10.3 Cancellation

11. Provider integration strategy

11.1 Provider breadth is not the goal

11.2 Why Anthropic is a preset, not a core native adapter

11.3 Registry model

11.4 Groq scope correction

12. Usage, cost, expiry and truth levels

12.1 Canonical source enum

12.2 What the wallet may show

12.3 Do not fake arbitrary credit math

12.4 Paid caps

12.5 Price data

13. Data model

13.1 Tables

13.2 Retention

14. Gateway contract and compatibility

14.1 Public local endpoints

14.2 Model names

14.3 Integration pattern

14.4 Useful response headers

14.5 Honest client compatibility

15. Security and privacy minimum

16. CLI

17. Testing strategy

17.1 Fake upstream

17.2 Unit tests

17.3 Integration matrix

17.4 Release acceptance

18. Build plan for one developer

18.1 Implementation priority inside each phase

18.2 Scope guard

19. Risks and deliberate trade-offs

20. Roadmap after MVP proves useful

v0.2 - Local polish and secret handling

v0.3 - Protocol compatibility

v0.4 - Broader inference surfaces

Later only if users ask

21. Architecture decision register

22. Recommended README / launch story

22.1 Tagline

22.2 One-line description

22.3 Demo story

22.4 Claims to avoid

23. Source reconciliation summary

From the v0.1 PDF [F1]

From the v0.2 Markdown [F2]

From the pasted feature proposal [F3]

24. Web research references

25. Final build checklist

Document purpose

This document reconciles three supplied artifacts and fresh competitor/provider research into one buildable MVP:

- **F1 - QuotaMesh MVP Architecture & Development Specification v0.1 (PDF):** the original local-first gateway skeleton.
- **F2 - QuotaMesh MVP Spec v0.2 - reviewed and revised (Markdown):** the primary architecture correction layer. It fixes quota grain, error classification, raw pass-through, key pools, security, usage rollups and visibility.
- **F3 - Pasted feature proposal:** the product-positioning layer that reframes QuotaMesh as an AI capacity wallet with project profiles, spend guardrails, an available-now view, route explanation, credit tracking and integration snippets.
- **Fresh web research checked 2 October 2026:** used to test whether the proposed differentiation is still real in the current gateway market.

The conflict rule is deliberate: **v0.2 wins over v0.1 on architecture correctness; the pasted feature list is adopted only where it adds strong user value without materially increasing implementation complexity; current competitor evidence can veto uniqueness claims.**

Executive decision

QuotaMesh should still be a **single-process, local-first developer tool**, not a general AI gateway platform. The core technical design in v0.2 is strong: FastAPI, one SQLite file, ordered deterministic routing, key pools, per-model quota state, raw OpenAI-compatible pass-through, passive health, explicit paid guardrails, and no mid-stream provider switching.

The product identity, however, should change one more time:

QuotaMesh is not “another LLM gateway.” It is a local AI capacity wallet for developers, with project-level allocation and safe failover.

That distinction matters because the 2026 market already has strong products for provider breadth, free-tier pooling, key rotation, per-model cooldowns, capability routing, observability and hosted budget enforcement. QuotaMesh should not try to beat those products feature-for-feature.

The final MVP has three primary user questions

1. What AI access do I own right now?

Free, trial/credit and paid credentials grouped honestly by account/project and current state.

2. What can this project safely use right now?

A project profile applies its own allowed targets, plan policy, expiry, cooldown and paid-spend rules.

3. Why will this request use that provider/key, or why is something blocked?

The same deterministic candidate function powers both live routing and the explanation UI.

Everything else should support those three questions.

What stays from the v0.2 architecture

- One Python process and one SQLite database.
- FastAPI + httpx + Jinja2/HTMX + Typer/Rich.
- Raw request pass-through; inspect only the small fields needed for routing.
- Provider registry data plus a generic OpenAI-compatible transport.
- Error classification from **status + headers + body**, not status alone.
- Key pools inside a target.
- Quota state at **(quota group, model)**, not blindly per API key.
- Paid access off by default.
- Fallback only before the stream commit point.
- Persistent quota/cooldown state and usage rollups.
- No prompt or response body logging by default.
- Localhost security that is cheap enough to belong in MVP.

What changes from the v0.2 Markdown

1. **Routes become Project Profiles in the product language.** Internally they are still ordered route configurations. Do not add a second project subsystem.
2. **The home page becomes the Capacity Wallet, not a live traffic dashboard.** Recent traffic is supporting evidence, not the product's primary mental model.
3. **"What can I use now?" ships, but capability-aware ranking does not.** MVP filters by profile policy, plan, expiry, health/cooldown and known model availability. Vision/tool/context metadata waits until later.
4. **The `private` auto-routing flag is deferred.** Show data-use notes now; automated privacy routing depends on policy metadata that can become stale.
5. **Anthropic and xAI are compatibility presets, not reasons to expand first-class provider code.** First-class support is defined by tested classifier behavior, not by a provider-count badge.
6. **No automatic pricing updater in MVP.** Provider-reported cost wins; a small verified price snapshot and optional user overrides are enough. Unknown cost remains unknown.
7. **No arbitrary "400 credits -> 274 remaining" math unless the credit unit is actually measurable.** Monetary credits can be locally estimated when cost is known; provider-specific abstract credit units are shown as manual/unknown.
8. **No background health race.** Use passive latency from real traffic and an explicit Doctor/Test action.
9. **Process-environment import may ship; parsing arbitrary `.env` files is deferred.** It is useful, but not necessary to prove the core value.
10. **Unknown 429s stay `COOLDOWN`, not confidently `EXHAUSTED`.** Do not infer a daily cap without evidence.

1. Competitive reality in October 2026

The biggest correction from the pasted proposal is that several apparently differentiating features are already present elsewhere. This does **not** kill the product, but it changes what should be emphasized.

1.1 What the current market already does well

Product / project	Current relevant capability	Implication for QuotaMesh
LiteLLM	Broad multi-provider router, load balancing, retries, fallbacks, cooldowns, routing strategies; Redis can track production cooldown/usage state. [W1]	Do not compete on routing sophistication or provider count.
LLM-API-Key-Proxy	Local universal gateway, key rotation, per-model cooldowns, OAuth discovery, usage tracking, key prioritization, TUI, OpenAI/Anthropic compatibility. [W2]	Per-model quota/key rotation is table stakes, not a unique hook.
freellmpool	18-provider free-tier pool, capacity status, provider health, free-provider discovery/checklist, dashboard, OpenAI-compatible proxy and a Responses shim. [W3]	“Capacity status”, health race and free API discovery are no longer unique by themselves.
Free LLM Gateway	24+ providers, 260+ free models, automatic fallback, round-robin, smart routing and a web dashboard. [W4]	Never chase free-provider breadth.
ModelMesh Lite	Free-tier aggregation, capability-based routing and multiple rotation strategies. [W5]	Capability/quality routing belongs out of MVP.
Vercel AI Gateway	Hosted unified gateway with fallbacks, routing rules, provider sorting by cost/latency/throughput, BYOK timeouts, budgets and policy controls. [W6][W7][W8]	Smart routing and budgets exist at cloud scale; QuotaMesh wins on local ownership and personal-project simplicity, not raw capability.
Portkey	Production gateway, fallbacks, load balancing, retries, observability, prompt management, caching and enterprise controls. [W9]	Avoid enterprise gateway scope completely.

1.2 The real gap QuotaMesh should target

The reviewed competitors largely optimize one of these goals:

- route across many providers/models;
- pool free access;
- rotate many keys;
- provide production observability/governance;
- centralize hosted billing.

QuotaMesh should optimize a different workflow:

I personally own a messy mix of free keys, trials, prepaid credits and paid keys. I have several local projects. I want one place to see that capacity, decide which projects may use which capacity, prevent accidental paid spend, and keep a project running when one allowed source is unavailable.

The differentiation is therefore **product focus and UX composition**, not a claim that no other gateway has any one of these primitives.

1.3 What not to market as unique

Do not lead with these claims:

- “We track capacity.” freeImpool already exposes capacity status. [W3]
- “We rotate keys per model.” LLM-API-Key-Proxy does more sophisticated per-model rotation/cooldowns. [W2]
- “We find free APIs.” freeImpool and Free LLM Gateway already surface free access. [W3][W4]
- “We route by latency/cost.” Vercel and others already do this dynamically. [W7]
- “We support lots of providers.” Multiple competitors support dozens. [W4]
- “We are OpenAI-compatible.” That is expected, not differentiating.

1.4 What to market instead

Lead with this sequence:

1. **Wallet:** all of *your* AI capacity in one local view.
2. **Project allocation:** the same credentials can have different rules for different local projects.
3. **Spend safety:** paid is off unless the selected project profile explicitly allows it; local daily/monthly caps are visible.
4. **Honest availability:** the UI distinguishes provider-reported, locally observed, manual and unknown information.
5. **Explainability:** every selection or skip is explainable with the same logic used by the gateway.
6. **Zero infrastructure:** no QuotaMesh account, Docker, Redis, Postgres or hosted control plane.

Recommended public positioning

QuotaMesh is your local AI API capacity wallet: connect the free, trial and paid credentials you already own, allocate them to project profiles, see what each project can use right now, and fail over safely without unexpected spend.

2. Target user and scenarios

2.1 Primary user

A solo developer or small technical builder who uses several AI APIs for local development, coding agents, experiments, prototypes and personal products. QuotaMesh must still be useful with **two credentials and one provider**; it must not require a complex multi-provider setup to justify installation.

2.2 Canonical mixed-capacity scenario

The user may own:

- Gemini project A - free access.
- Gemini project B - free access.
- Groq project - free/developer access.
- NVIDIA NIM developer access.
- OpenAI prepaid/paid access.
- Anthropic or xAI paid/trial access through compatibility mode.
- A hackathon/student/promotion credential with an expiry date.

The user also has several projects with different risk tolerances:

- **portfolio-builder** - free and trial only; never paid.
- **voice-agent** - free first, then paid OpenAI up to a small daily cap.
- **experiments** - any configured source in explicit order.
- **client-work** - only selected paid accounts the user has deliberately approved.

QuotaMesh centrally holds the credentials, but each project profile controls which of them can be considered.

2.3 Failure scenario

A request for `qm/voice-agent` resolves to an ordered profile:

1. Gemini / model A / key pool [Personal, Work]
2. Groq / model B / key pool [Dev]
3. OpenAI / model C / key pool [Paid]

If Gemini Personal is cooling for that model, Gemini Work is tried. If all Gemini keys in that quota group are exhausted, Groq is tried. The paid OpenAI target is considered only if the profile has `allow_paid=true` and its local cap has not been reached. Every skipped candidate records a reason.

2.4 Important compliance boundary

QuotaMesh manages credentials the user is authorized to use. It does not create accounts, harvest identities, bypass provider controls, or advertise “quota multiplication.” This is especially important because Groq’s current acceptable-use policy explicitly prohibits exceeding published limits by registering multiple accounts or orchestrating usage between multiple organizations. [W15]

3. MVP product principles

Principle	Operational meaning
Local-first	No QuotaMesh cloud account. Configuration, secrets and observed usage stay on the machine.
Wallet-first	The primary UI answers “what capacity do I own?” before it answers “what models exist?”
Project-safe	Every app uses a named project profile with explicit allowed targets and spend rules.
Predictable over clever	The user defines order. QuotaMesh filters and fails over; it does not invent a better model.
Observed, not invented	Unknown quota/balance stays unknown. Local estimates are visibly labeled.
Free-safe by default	Paid credentials are excluded unless the project profile explicitly permits them.
Passive health	Real traffic updates health and latency. Explicit Doctor runs probes; there is no background quota burner.
One process	Gateway, dashboard, registry, routing core and persistence run together.
Small provider surface	A generic transport plus small classifier hooks; provider rows are data, not a growing adapter hierarchy.
No mid-stream magic	Fallback is allowed only before response commitment. Never splice providers into one emitted stream.
Honest compatibility	Ship Chat Completions first and state clearly which tools require Responses or Messages later.
Provider terms still apply	QuotaMesh is a manager, not a circumvention layer.

4. Final MVP scope

4.1 Ship in v0.1

A. Product-facing features

1. AI Capacity Wallet

- Group access into FREE, TRIAL/CREDIT and PAID.
- Show health, last success, last error, expiry, observed usage and spend/credit truth level.
- Group multiple secrets that share a quota project so the wallet does not double-count capacity.

2. Add/Test Credentials

- Provider, secret, plan type required.
- Optional account/project label, quota-group label, expiry and starting monetary credit.
- Test and Save uses model listing where practical; otherwise permits Save as Untested.

3. Project Profiles

- Product-facing replacement for generic “routes.”
- Ordered targets, key pools, paid/trial flags, attempt budget, deadline and optional paid caps.
- The backend does **not** need a separate project subsystem; the profile is the route.

4. Automatic Health-Aware Fallback

- Key pools per target.
- Per-(quota group, model) cooldown/exhaustion state.
- Body-aware error classification.
- No failover after stream commit.

5. Paid/Trial Guardrails

- `allow_paid=false` by default.
- Optional daily and monthly local paid caps.
- Trial expiry countdown and automatic exclusion after expiry.

6. “Available Now” for a Project

- Run the candidate function without making an upstream call.

- Show eligible candidates and blocked candidates with reasons.
- MVP filters on profile policy, plan, expiry, health, quota state and known model access.
- It does **not** rank model quality or maintain a large capability database.

7. Explain Route / Dry Run

- Same selection code as live traffic.
- Show ordered candidates, skip reasons, cap decisions and earliest recovery time.

8. One-Click Integration Snippets

- Python OpenAI SDK, Node OpenAI SDK, cURL, environment variables and OpenCode.
- Use one stable `/v1` base URL and `model="qm/<profile>"`; do not invent profile-specific HTTP paths.

9. Truth Labels for Usage/Quota

- Backend source enum: `PROVIDER`, `LOCAL`, `MANUAL`, `UNKNOWN`.
- UI labels can render as Live / Observed / Manual / Unknown.
- A permanent notice states that local counts only include traffic through QuotaMesh.

B. Small supporting utilities that are worth shipping

- **Doctor:** on-demand key/model test and latency result.
- **Recent Activity / History:** request attempts and fallbacks, no prompt bodies.
- **Demo:** a fake upstream used by tests that can demonstrate failover without real keys.
- **Access Catalog:** a curated, data-only list of provider signup/docs links and whether the user has configured them. No promises about exact free limits.
- **Environment import:** import keys already present in the running process environment. Arbitrary `.env` file parsing is later.

4.2 Stretch only if the core is complete

- Rotate/round-robin strategy in addition to sticky priority.
- CSV export.

- Shared-quota hint when multiple keys repeatedly fail together.
- Simple 14-day sparkline rendered server-side.
- Copy buttons for more frameworks.
- User-entered per-key local daily request budget.

4.3 Deferred to v0.2+

- `/v1/responses` translation.
- Native `/v1/messages` translation.
- Embeddings, images, audio/video normalization.
- Model capability database for tool/vision/context filters.
- Automatic latency/cost optimization.
- OS keyring / encrypted secret store.
- `.env` file scanning/import from arbitrary project directories.
- Textual full-screen TUI.
- Notifications and alerts.
- Hosted sync or team features.
- Native desktop shell.
- OAuth account discovery.
- Automatic live pricing updater.
- Provider-specific billing integrations.

4.4 Explicitly reject from MVP

- Semantic or AI-powered routing.
- Model quality ranking.
- Prompt management.
- Caching.
- RAG, memory, MCP or agent orchestration.
- PostgreSQL, Redis, Kafka or distributed workers.
- RBAC, SSO, teams or organization admin.
- Provider account creation.
- Free-tier circumvention automation.
- 40+ provider race.

4.5 Feature decision matrix from the pasted proposal

Proposed feature	Final decision	Reason
AI Capacity Wallet	SHIP - headline	High value and cheap because data already exists.
Project Profiles	SHIP - headline	Reuses route tables; strong project-level differentiation.
Free/Trial/Paid Guard	SHIP - headline	Core trust feature; very small selection logic.
What can I use now?	SHIP - simplified	Reuse candidate engine; no capability/quality ranking in v0.1.
One-click integration	SHIP	Template output, almost no backend complexity.
Explain My Route	SHIP	Candidate engine already knows every skip reason.
LIVE/ESTIMATED/UNKNOWN	SHIP, with stricter names	Preserve provider/local/manual/unknown internally to avoid ambiguity.
Trial/Credit tracker	SHIP - monetary estimate only when measurable	Do not invent conversions for arbitrary provider credit units.
Health + latency race	SHIP as passive + manual Doctor	No background polling or quota burn.
Free API discovery	SHIP as small Access Catalog	Useful onboarding, but not unique enough to be a core differentiator.
Fallback Simulator	SHIP as dry-run + demo	Reuse pure engine and fake upstream; no elaborate simulator subsystem.
.env file importer	DEFER	Nice but not core; environment import is enough initially.
Alerts/notifications	DEFER	Adds background/scheduling complexity.
AI-powered routing	REJECT	Hard, already crowded and contrary to predictable-product thesis.

5. User experience and information architecture

The MVP should feel like a small developer utility, not a miniature cloud console.

5.1 First-run flow

```
uvx quotamesh start
  |
  v
Browser opens to one-time local login URL
  |
  v
Add API access
(provider + key + plan)
  |
  v
QuotaMesh tests it and auto-creates qm/default
  |
  v
Capacity Wallet appears
  |
  v
Create Project Profile
  |
  v
Choose allowed targets / paid rule / cap
  |
  v
Available Now + Explain Route
  |
  v
Copy integration snippet
  |
  v
Run project through http://127.0.0.1:8787/v1
```

5.2 Screen 1 - Capacity Wallet (home)

The home screen should **not** lead with routers/providers/models. It should lead with owned capacity.

MY AI CAPACITY

FREE

```

-----
Gemini / Personal Project    Healthy    482 ms
  gemini-...                 OK         Observed: 183 req today
Gemini / Work Project        Cooling    retry in 24s
Groq / Dev                   Healthy    231 ms

```

TRIAL / CREDITS

```

-----
Hackathon API               Expires in 19 days
  Starting credit: $20       Observed spend: $6.40
  Est. remaining: $13.60    [LOCAL ESTIMATE]

```

PAID

```

-----
OpenAI / Personal           Healthy
  Observed spend today: $0.23 [LOCAL]
  Default use: BLOCKED unless project allows paid

```

Wallet design rules

- A **credential** is a secret. A **capacity source** is what the user thinks they own: provider + account/project/quota group, optionally split by model state.
- If two secrets share one quota group, show one capacity source with “2 keys” rather than pretending the user owns double quota.
- Every quantity has a source badge.
- Show last checked/last successful time so stale health is obvious.
- Do not show a numeric “remaining” value if QuotaMesh cannot justify it.

5.3 Screen 2 - Project Profiles

A Project Profile is a named routing policy for one local app or workload.

```
VOICE-AGENT

Mode: Free first, paid backup
Paid fallback: ON
Daily paid cap: $2.00
Monthly paid cap: $15.00

ORDER
1. Gemini / model A / [Personal, Work]
2. Groq / model B / [Dev]
3. OpenAI / model C / [Paid]

[Available now] [Explain route] [Copy integration]
```

Profiles use explicit ordered targets. Templates are only form helpers; after creation, the stored order is authoritative.

Profile templates

- **Free only:** paid and trial disabled.
- **Free + trial:** free targets first, trial targets later, paid disabled.
- **Free first, paid backup:** free/trial targets first, paid last, paid explicitly enabled.
- **Advanced ordered:** exactly the order the user enters.

There is no hidden reordering at request time.

5.4 Screen 3 - Available Now

This is a **policy-aware availability view**, not a model recommender.

```
AVAILABLE NOW FOR: voice-agent

ELIGIBLE
1. Gemini / model A / Work      FREE      Healthy
2. Groq / model B / Dev        FREE      Healthy
3. OpenAI / model C / Personal PAID      Healthy - allowed, cap room $1.77

BLOCKED / SKIPPED
Gemini / Personal              Cooling - retry in 24s
Trial API                      Expired yesterday
OpenAI / Backup                Paid - not listed in this project
```

MVP filters:

- selected profile targets;
- enabled/disabled state;
- expiry;
- plan permission;

- quota-group/model cooldown/exhaustion;
- credential invalidity;
- local paid cap;
- known model availability.

Deferred filters: tool calling, vision, audio, context window, quality class and semantic task type. Those require a maintained capability catalog and move the product back toward gateway competition.

5.5 Screen 4 - Keys

Normal form fields:

- Provider.
- API key / environment reference.
- Plan: FREE / TRIAL_CREDIT / PAID / UNKNOWN.
- Label.

Advanced fields:

- Account/project label.
- Quota group.
- Trial expiry.
- Starting monetary credit (USD when applicable).
- Priority inside provider pool.
- Custom base URL.
- Optional price override for an otherwise unknown paid model.

Do not ask for RPM/TPM/RPD or context window in the normal flow.

5.6 Screen 5 - Activity / History

A compact request list with one expandable attempt trace:

```
12:03:41 voice-agent 200 821 ms 2 attempts
  1. Gemini / Personal SKIP cooldown until 12:04:05
  2. Gemini / Work 200 821 ms
```

Store metadata only. No prompt or response bodies.

5.7 Screen 6 - Doctor

On user request:

Gemini / Personal	OK	482 ms	models discovered: 7
Gemini / Work	429		retry after 24s
Groq / Dev	OK	231 ms	rate headers seen
OpenAI / Paid	OK	640 ms	paid use disabled in default profile

Doctor may use a model-list call where supported. If a generation request is required, ask before spending quota or allow “save as untested.”

5.8 Screen 7 - Access Catalog

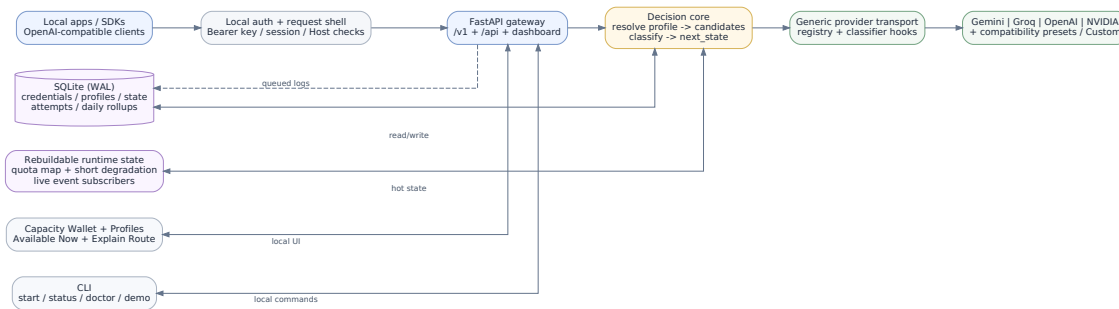
A deliberately small onboarding/catalog screen:

Provider	Access note	Configured	Action
Gemini	Free/dev access available	Yes	Docs
Groq	Developer access	Yes	Docs
NVIDIA NIM	Developer endpoints	No	Get access
Cerebras	Developer access	No	Get access
OpenRouter	Free models may be listed	No	Open site

Rules:

- Registry data only, no scraper.
- Store docs/signup URL and `last_verified` date.
- Do not hardcode volatile request/day numbers into routing logic.
- Phrase availability cautiously; provider programs change.

6. System architecture



QuotaMesh system architecture

6.1 Process model

One `uvicorn` worker runs the entire local application:

```

quotamesh process
|
+-- FastAPI
|   +-- dashboard routes
|   +-- /api/* local management API
|   +-- /v1/* gateway API
|   +-- /events recent activity SSE
|
+-- Decision core (pure functions)
|   +-- resolve_profile()
|   +-- candidates()
|   +-- classify()
|   +-- next_state()
|
+-- Generic provider transport
|   +-- registry rows
|   +-- small provider classifier hooks
|
+-- Shared httpx.AsyncClient
|
+-- SQLite repository (WAL)
|
+-- In-memory runtime state
|   +-- active quota states
|   +-- short provider/model degradation
|   +-- recent event subscribers
  
```

No microservices. No Docker requirement. No external queue. No Redis. No Postgres.

6.2 Technology choices

Layer	MVP choice	Why
Language	Python 3.11+	Fast iteration and strong async/library ecosystem.
Web/gateway	FastAPI + Uvicorn, one worker	Simple local server; one process keeps state deterministic.
Upstream HTTP	One shared <code>httpx.AsyncClient</code> , redirects disabled	Connection reuse and prevents keys being redirected to another host.
Persistence	<code>stdlib sqlite3</code> , WAL, short transactions	Zero setup; enough for a single-user local tool.
UI	Jinja2 + HTMX + minimal JavaScript	No SPA/bundler/build pipeline.
Live activity	Small SSE endpoint or HTMX polling	Cheap visibility; not a full observability stack.
CLI	Typer + Rich	Good local developer ergonomics.
Validation	Pydantic for local config/registry only	Do not model the full upstream request schema.
Packaging	<code>uv</code> / PyPI / <code>uvx quotamesh</code>	One-command onboarding.
Timezones	<code>zoneinfo</code> + <code>tzdata</code> dependency on Windows	Correct provider reset boundaries without custom timezone code.

6.3 Recommended source layout

```
quotamesh/  
  app.py  
  cli.py  
  config.py  
  store.py  
  
  engine/  
    select.py      # profile resolution + candidates  
    classify.py     # error classification  
    state.py       # quota state transitions  
    stream.py      # commit-point logic  
    usage.py       # accounting / rollups  
  
  providers/  
    transport.py   # generic OpenAI-compatible transport  
    classifiers.py  # small named classifiers only where needed  
  
  registry/  
    providers.toml  
    prices.json    # small verified snapshot, optional overrides  
  
  ui/  
    templates/  
    static/  
  
  demo/  
    fake_upstream.py  
  
  tests/  
    fixtures/  
    integration/
```

6.4 Functional core, imperative shell

Keep correctness-critical logic pure:

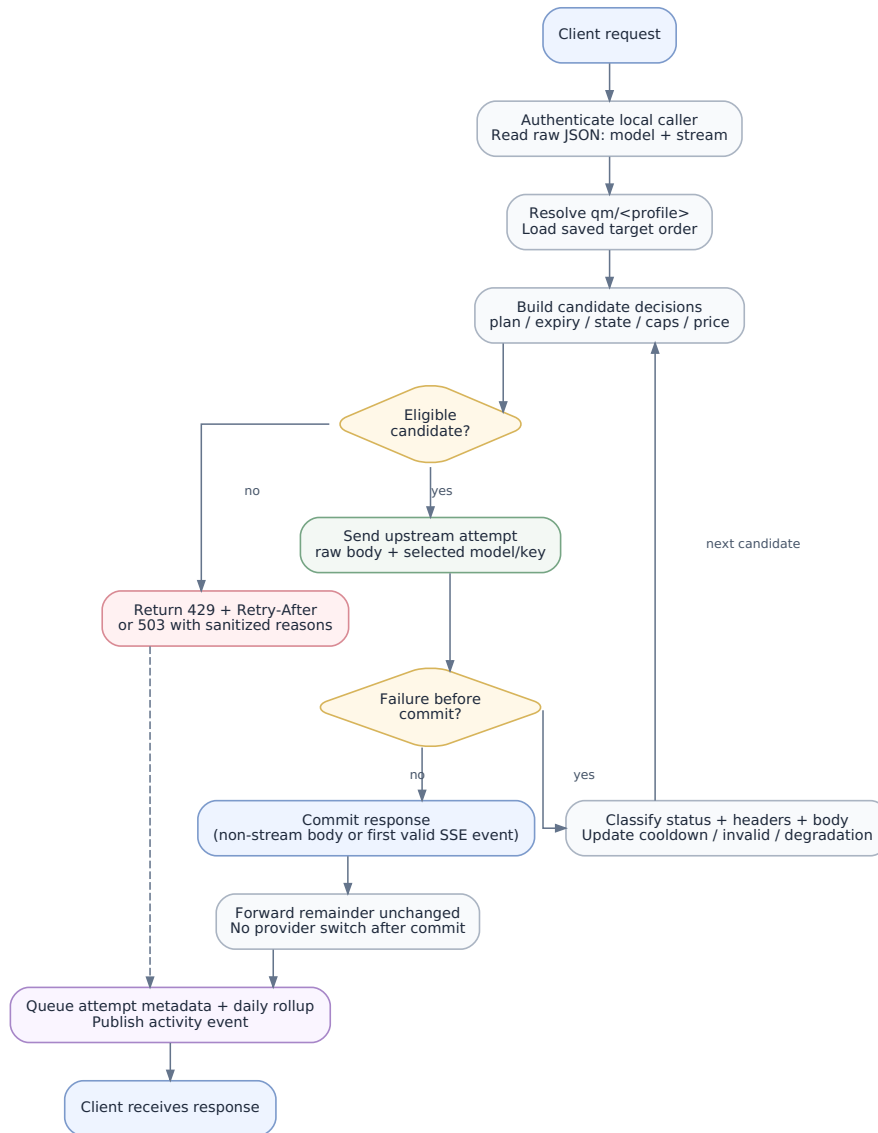
```
classify(provider, status, headers, body) -> Outcome  
next_state(current, outcome, now, registry) -> State  
candidates(profile, credentials, states, usage, now) ->  
    list[CandidateDecision]
```

The same `candidates()` result is used by:

- live traffic;
- Available Now;
- Explain Route;
- fallback simulator/dry-run;
- tests.

This eliminates a common failure mode where the UI explains one policy while production routing applies another.

7. Request lifecycle and routing engine



Request decision flow

7.1 Gateway flow

```
client
-> local bearer/session checks
-> read raw JSON
-> read only model + stream (+ minimal routing facts)
-> resolve qm/<profile> or direct provider/model target
-> build candidate decisions
-> attempt eligible candidates in deterministic order
-> classify outcome
-> update state
-> next candidate OR commit response
-> write attempt metadata + daily rollup
-> publish activity event
```

7.2 Raw pass-through rule

Do **not** define a full OpenAI Chat Completions Pydantic request model. Provider-compatible APIs evolve and may accept extension fields. A strict local schema can silently drop fields or reject requests that an upstream would accept.

Implementation rule:

- parse JSON once;
- require/inspect `model` and `stream`;
- replace the profile alias with the selected upstream model;
- apply only explicit provider quirk hooks;
- forward the rest of the JSON.

7.3 Core objects

Credential

A concrete secret plus ownership metadata:

```
id
provider_id
label
plan_type
account_label
quota_group
priority
expiry
starting_credit_usd (optional)
base_url (custom only)
status
enabled
```

Project Profile

A named route/policy used by an application:

```
name / slug
ordered targets
allow_trial
allow_paid
max_attempts
nonstream deadline
first-event timeout
paid daily cap
paid monthly cap
```

Target

```
provider + model + key pool
```

A null pinned key means “use all enabled keys for this provider in configured priority order.”

Candidate Decision

The engine should return eligible **and skipped** candidates:

```
candidate = provider/model/key
eligible = true|false
skip_reason = cooldown|expired|paid_blocked|cap_reached|invalid|...
recovery_at = optional timestamp
```

That result is the foundation of explainability.

7.4 Selection algorithm

```
for target in profile.targets in saved order:
    for key in target.pool in sticky priority order:
        skip if disabled or expired
        skip if credential INVALID/UNUSABLE
        skip if TRIAL and profile disallows trial
        skip if PAID and profile disallows paid
        skip if quota_state(quota_group, model) is cooling/exhausted
        skip if paid cap reached
        skip if cost unknown while a paid cap requires known cost
        yield eligible candidate
```

Important properties:

- **No AI router.**
- **No runtime “free first” reordering.** Templates can create an initial order, but saved order is final.

- **Skipped candidates do not consume** `max_attempts` . Only real upstream calls count.
- **No same-key retry in v0.1.** If a request fails before commit, move to the next eligible candidate. Let the client retry a fresh request if nothing remains. This is simpler and avoids hidden duplicate work.
- **No waiting minutes inside one client request.** A cooldown means skip now, not sleep.

7.5 Attempt limits

Recommended defaults:

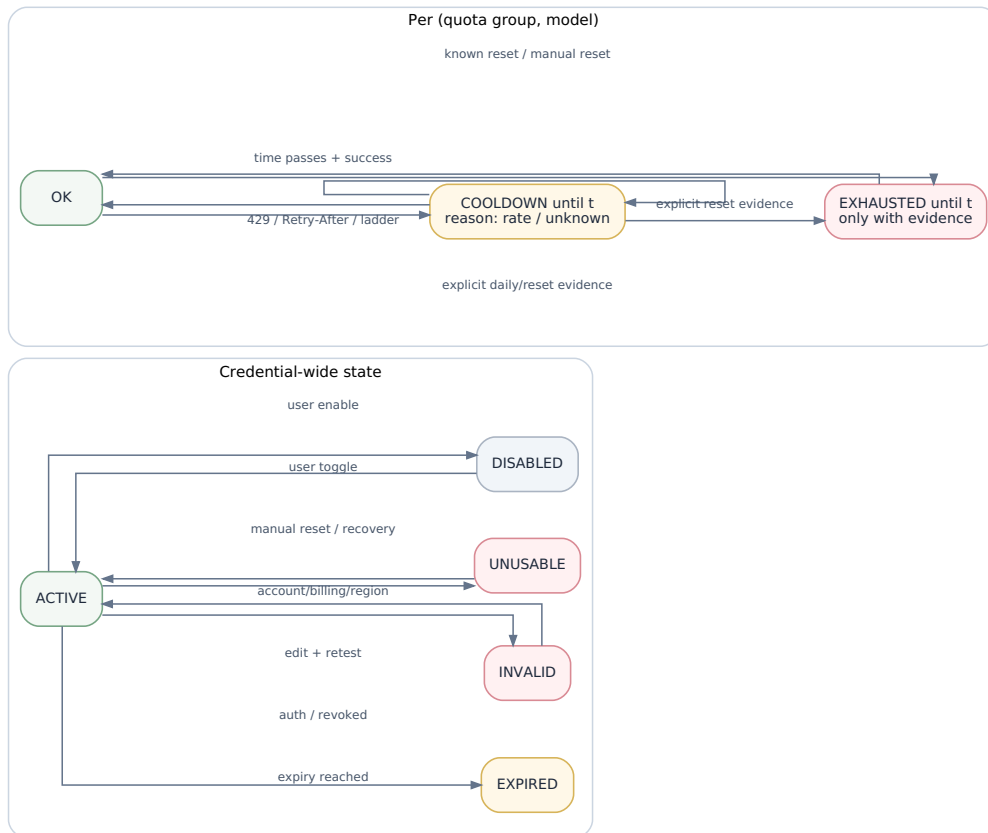
Setting	Default	Notes
max_attempts	5	Enough for several free sources plus a paid backup without an unbounded chain.
nonstream deadline	45 s	Total profile attempt budget for non-streaming.
connect timeout	5 s	Per upstream connection.
first-event timeout	30 s	For streams; user may raise for slow reasoning models.
idle stream timeout	60 s	Time between stream events.

7.6 When all candidates are blocked

- If the nearest known recovery is within about 60 seconds, return a `429` with `Retry-After` .
- Otherwise return a `503` with an OpenAI-shaped error and a sanitized `quotamesh` object explaining blocked candidates.
- Never reveal full keys in error details.

8. Health and quota state model

Timeouts / 5xx / model-access blocks are short-lived in-memory degradation, not persistent quota exhaustion.



QuotaMesh state model

The v0.1 PDF tracked state too coarsely. The v0.2 Markdown correctly moved quota state to `(quota group, model)`. The final MVP keeps that correction but simplifies target-level edge cases.

8.1 Persisted state scopes

Credential-wide state

Stored on the credential row:

- ACTIVE
- INVALID
- UNUSABLE
- EXPIRED
- DISABLED

Used for failures that genuinely apply to the secret/account, not one model.

Quota-group/model state

Persisted in `quota_state`:

- OK
- COOLDOWN until `t`
- EXHAUSTED until `t`

Keyed by `(quota_group, model)` so keys from the same provider project can share an actual rate/quota boundary.

Short-lived target/provider degradation

Kept in memory only:

- temporary provider/model `DEGRADED` after timeout/5xx;
- short target block for a model-access 403/404.

These states last seconds/minutes and do not justify more schema in v0.1. A restart clears them, which is an acceptable MVP trade-off.

8.2 Unknown 429 cooldown ladder

When a provider gives no reliable reset metadata:

```
5 s -> 15 s -> 60 s -> 5 min -> 30 min -> 2 h
```

A success resets the strike count.

Do not label the credential “daily exhausted” just because three 429s occurred. Keep the state as `COOLDOWN (unknown limit)` unless the body/header/provider rule gives evidence for a daily/resettable cap.

8.3 Explicit information wins

Use provider reset metadata when available:

- `Retry-After` for temporary limits.
- remaining/reset headers when documented.
- Gemini project/model RPD reset rule when a daily limit is clearly identified. Google’s current rate-limit docs say limits are applied

per project, vary by model, and daily request quotas reset at Pacific midnight. [W10]

- OpenAI documents request/token rate-limit headers and project-scoped token headers. [W11]

The registry may contain reset rules, but live provider evidence wins over static metadata.

9. Failure classification matrix

Error handling is one of the most important architecture corrections from v0.1. Status code alone is not sufficient; for example, provider APIs can return authentication or quota semantics inside different 4xx shapes.

Signal	Meaning	Action	State update
401 auth failure	Secret rejected	Next candidate	Credential INVALID until edit/retest.
Gemini 400 body clearly says invalid/ expired key	Secret rejected	Next candidate	Credential INVALID .
Error says leaked/ revoked key	Secret unusable	Next candidate	Credential INVALID ; UI says rotate.
403 model access	Key/account lacks model access	Next candidate	Short target block; do not invalidate whole credential unless body says account/ billing/region.
402 / insufficient quota / payment required	Paid/trial funds unavailable	Next candidate	Credential or quota source UNUSABLE/ EXHAUSTED until user reset/top-up.
429 + short Retry-After	Temporary rate limit	Next candidate	COOLDOWN until reset.
429 + explicit daily/ reset wording	Daily quota exhausted	Next candidate	EXHAUSTED until known reset.
429 with no useful info	Unknown limit	Next candidate	Cooldown ladder only.
404 model not found	Stale/misspelled target	Next target	Short target block; Doctor flags route.
400 context length / 413	Prompt too large	Next target if user explicitly configured one	No key cooldown.
Other 400 / 422	Malformed/ unsupported request	Return upstream error	No fallback by default.
Timeout/connection before commit	Network/provider transient	Next candidate	Provider/model degraded briefly.
5xx / overload before commit	Provider transient	Next candidate	Provider/model degraded briefly.
200 whose first SSE event is an error object	Failure before commit	Classify and fall back	According to contained error.

Signal	Meaning	Action	State update
Protocol/malformed response before commit	Upstream protocol failure	Next candidate	Short degradation.
Failure after stream commit	Cannot splice provider	Close with error event	Log <code>MID_STREAM_FAILURE</code> ; no transparent fallback.
Client disconnect	User canceled	Cancel upstream	Log <code>CLIENT_CANCELLED</code> .
Safety/content refusal	Valid model result	Return as-is	No routing penalty.
Paid cap reached	Local guard	Skip candidate	No provider state change.
Unknown price while cap is required	Cannot guarantee cap	Skip paid candidate by default	No provider state change.
SQLite log write failure	Local observability failure	Keep serving request	Increment diagnostic counter.

10. Streaming rules

The commit point must be exact and simple.

10.1 Commit point

For a streaming request:

1. Open upstream stream.
2. If HTTP status is non-2xx, classify and try the next candidate.
3. If 2xx, read the first **complete SSE event**.
4. If that first event is an error object, classify and fail over.
5. Otherwise commit the chosen provider by sending headers + first event to the client.
6. After commit, forward raw bytes/events; **never switch provider**.

A role-only first chunk counts as commitment. Do not wait for visible text.

10.2 Usage capture

- Non-streaming: parse the final JSON for usage/cost metadata, but return the original upstream bytes where practical.
- Streaming: only inject `stream_options.include_usage` for providers explicitly marked compatible in registry data.
- Missing usage remains null; never estimate tokens silently.
- Keep only the minimal tail needed to parse final usage metadata.

10.3 Cancellation

If the client disconnects:

- cancel/close the upstream response immediately;
- stop reading tokens the user will never receive;
- log `CLIENT_CANCELLED`.

11. Provider integration strategy

11.1 Provider breadth is not the goal

The MVP should have a **small tested core plus compatibility presets**, because provider count is already an established competitor strength.

First-class tested providers

These deserve provider-specific classifier fixtures and release smoke tests:

- Google Gemini.
- Groq.
- OpenAI.
- NVIDIA NIM.

Compatibility presets

These may be registry rows using the generic OpenAI-compatible transport. They are useful, but v0.1 should not promise rich quota parsing until tested:

- xAI.
- Anthropic OpenAI compatibility.
- OpenRouter.
- Cerebras.
- Mistral.
- Other OpenAI-compatible endpoints.

Custom

User supplies base URL + key + model. Generic behavior only.

11.2 Why Anthropic is a preset, not a core native adapter

Anthropic's current OpenAI SDK compatibility layer is explicitly described as primarily for testing/comparison rather than the best long-term production interface, and it omits or changes some native capabilities such as prompt caching. [W12] That makes it suitable as a compatibility preset, while a native Messages endpoint can come later.

11.3 Registry model

Example:

```
[gemini]
name = "Google Gemini"
base_url = "https://generativelanguage.googleapis.com/v1beta/openai/"
quota_scope = "project"
limit_grain = "per_model"
reset_rule = "midnight_pt"
classifier = "gemini"
supports_model_list = true
last_verified = "2026-10-02"
```

Registry rules:

- Store stable behavior, not a giant model catalog.
- Ship no hardcoded “free requests/day” numbers as routing truth.
- Model IDs are discovered or typed by the user.
- Provider metadata has `last_verified`.
- Base URL redirects are never followed when secrets are attached.
- Quirk hooks are explicit and opt-in: strip a known unsupported parameter, inject stream usage, rewrite a model prefix.

Google currently documents an OpenAI-compatible Gemini endpoint, but labels the compatibility support as beta. [W13]

11.4 Groq scope correction

The v0.2 Markdown described Groq primarily at organization/model grain. Current Groq project documentation says API keys are project-specific and projects may have their own rate limits under an organization ceiling; some token limits remain organization-wide. [W14] Therefore QuotaMesh should not hardcode one simplistic scope. The practical MVP rule is:

- default each Groq key to a unique quota group;
- allow the user to group keys if they know they share a limit;
- use actual headers/errors as authority;
- document that an organization ceiling can still cause seemingly independent projects to fail together.

This is another reason the state model should be evidence-driven, not based on a static quota formula.

12. Usage, cost, expiry and truth levels

The wallet is only trustworthy if it is explicit about what it knows.

12.1 Canonical source enum

Source	Meaning	UI label example
PROVIDER	Directly reported by the provider in headers/body/API.	LIVE
LOCAL	Counted or estimated only from traffic through QuotaMesh.	OBSERVED / LOCAL ESTIMATE
MANUAL	User-entered amount, limit or expiry.	MANUAL
UNKNOWN	Cannot be known reliably.	UNKNOWN

Do not replace these backend meanings with one fuzzy `ESTIMATED` enum.

12.2 What the wallet may show

- Requests through QuotaMesh - exact local count.
- Input/output tokens - provider usage field when returned.
- Gateway latency/TTFB - local measurement.
- Remaining RPM/TPM - provider headers when available; otherwise unknown/manual.
- Paid spend - provider-reported cost when available, otherwise local price estimate if price known.
- Trial expiry - manual date.
- Starting monetary credit - manual input.
- Estimated monetary remaining - starting monetary credit minus observed/estimated spend, labeled local estimate.

12.3 Do not fake arbitrary credit math

If a provider grants “400 credits” but does not expose what one credit means per request/token/model, QuotaMesh must not subtract request count from the credit number and claim “274 remaining.” Instead show:

```
Starting provider credits: 400      [MANUAL]
Observed requests: 126             [LOCAL]
Remaining provider credits: UNKNOWN
Expires: 2026-12-31               [MANUAL]
```

If the provider later exposes machine-readable credit consumption, add a provider parser.

12.4 Paid caps

Before a paid candidate is selected:

1. aggregate observed spend for the profile/day/month;
2. compare with configured caps;
3. if cap reached, skip;
4. if cost is unknown and a dollar cap is configured, skip by default unless the user explicitly allows unknown-price usage.

Local caps are safety rails, not provider billing controls. In-flight requests can overshoot by their own final cost.

12.5 Price data

MVP order of trust:

1. provider-reported request cost;
2. small bundled price snapshot with `last_verified`;
3. user override;
4. unknown.

Do not build a pricing crawler or license-sensitive third-party pricing sync before MVP proves useful.

13. Data model

Eight small tables are enough. Use `PRAGMA user_version` plus explicit migration functions; do not add Alembic.

13.1 Tables

credentials

id	
provider_id	
label	
plan_type	FREE TRIAL_CREDIT PAID UNKNOWN
account_label	
quota_group	
priority	
starting_credit_usd	nullable
trial_expires_at	nullable
base_url	custom only
fingerprint	hash prefix / duplicate detection
status	
status_reason	
enabled	
created_at	
updated_at	

secrets

credential_id	
secret_value	nullable if env reference
env_name	nullable

The management API never returns `secret_value`.

quota_state

quota_group	
model	
status	OK COOLDOWN EXHAUSTED
until	nullable
strikes	
reason	
updated_at	
PRIMARY KEY (quota_group, model)	

project_profiles

```
id
name
slug
allow_paid
allow_trial
max_attempts
nonstream_deadline_s
first_event_timeout_s
paid_daily_cap_usd
paid_monthly_cap_usd
enabled
```

profile_targets

```
profile_id
position
provider_id
model
credential_id          nullable = use provider pool
```

attempts

```
id
ts
request_id
profile_id
client_label
attempt_idx
provider_id
model
credential_id
outcome
http_status
error_class
fallback_reason
ttfb_ms
latency_ms
streamed
input_tokens
output_tokens
provider_cost_usd
estimated_cost_usd
skipped_json          first attempt row only
```

daily_usage

```
day
profile_id
credential_id
model
requests
failures
input_tokens
output_tokens
cost_usd
PRIMARY KEY (day, profile_id, credential_id, model)
```

This key lets the UI aggregate by project, credential or global wallet without losing monthly cap data when old attempt rows are pruned.

settings

```
key
value
```

Examples: port, display timezone, retention, local API-key hash, schema options.

13.2 Retention

- Keep attempts for 30 days or 50,000 rows by default.
- Keep daily rollups much longer.
- Pruning attempts must never change paid caps or wallet totals.
- Never store prompt/response bodies by default.

14. Gateway contract and compatibility

14.1 Public local endpoints

Endpoint	v0.1
POST /v1/chat/completions	Required; streaming and non-streaming.
GET /v1/models	Profile aliases plus discovered models.
GET /health	Process health only.
GET /events	Local recent-activity stream, authenticated.
/api/*	Local management API, authenticated.
POST /v1/responses	Deferred.
POST /v1/messages	Deferred.

14.2 Model names

Recommended profile alias:

```
qm/voice-agent
qm/portfolio-builder
qm/experiments
```

Optional direct target:

```
provider/model-name
```

The first credential can auto-create `qm/default` with paid disabled.

14.3 Integration pattern

Use one base URL:

```
from openai import OpenAI

client = OpenAI(
    base_url="http://127.0.0.1:8787/v1",
    api_key="<local-quotamesh-key>",
)

response = client.chat.completions.create(
    model="qm/voice-agent",
    messages=[{"role": "user", "content": "Hello"}],
)
```

Do not use `/v1/<project>` path tricks. Keeping the normal OpenAI API shape maximizes compatibility and keeps project selection in the model alias, where SDKs already expect a model identifier.

14.4 Useful response headers

On committed responses:

```
X-QuotaMesh-Profile
X-QuotaMesh-Provider
X-QuotaMesh-Model
X-QuotaMesh-Key           # label only
X-QuotaMesh-Attempts
X-QuotaMesh-Fallback
X-QuotaMesh-Request-Id
```

Optional request header:

```
X-QuotaMesh-Client
```

14.5 Honest client compatibility

Expected to work after smoke testing:

- OpenAI Python/JS SDKs using Chat Completions.
- LangChain/LangGraph when configured with a custom OpenAI-compatible base URL.
- Aider/editor tools that allow an OpenAI-compatible custom provider.
- OpenCode supports custom provider base URLs, making a generated config snippet valuable. [W16]

Not a v0.1 promise:

- **Codex CLI:** current Codex custom provider flows center on the Responses wire API; OpenAI's own repository includes a Responses API proxy configured with `wire_api='responses'`. [W17] A Chat Completions-only QuotaMesh should not claim native Codex support.
- **Claude Code:** native gateway support requires the Anthropic Messages protocol, not merely Chat Completions.

This limitation is acceptable for the MVP because adding protocol translation is a materially larger correctness surface than the wallet/profile product value.

15. Security and privacy minimum

A local gateway holding paid API keys is still a credential service. The following defenses are cheap enough that deferring them would be false economy.

Risk	MVP defense
Malicious webpage calls localhost	Bind only to <code>127.0.0.1</code> ; require random local bearer key on <code>/v1</code> ; no CORS; validate Host; reject foreign Origin/Sec-Fetch-Site on state-changing dashboard/API calls.
Dashboard token in URL leaks	Startup prints a one-time bootstrap URL; server sets <code>HttpOnly/SameSite=Strict</code> session cookie and redirects to a clean URL.
Secret redirected to another host	<code>httpx</code> redirects disabled; each provider key only sent to configured provider base URL.
Secret exposed by API/UI	Full secret is write-only after save; UI shows masked label only.
Secret in logs	Redact Authorization/api-key headers; disable verbose <code>httpx</code> body logging; automated test scans logs for key fragments.
Local disk theft/backups	Plaintext secret storage is an accepted MVP trade-off; isolate secrets table; use user-profile data directory; best-effort restrictive permissions; allow environment-only secret references.
Prompt leakage	Store no request or response bodies.
Hidden telemetry	None in v0.1; document this.
Cross-account abuse positioning	Product wording is neutral; no account creation or “multiply limits” language; show provider terms note.

Platform note

Unix-like systems can use directory mode `0700` and database mode `0600`. On Windows, do not pretend POSIX mode bits provide the same guarantee; keep the default data directory under the user’s profile and document the plaintext-key trade-off until native credential storage is added.

16. CLI

Keep the terminal surface small and task-oriented:

Command	Purpose
<code>quotamesh start</code>	Start gateway/dashboard and print local URL.
<code>quotamesh status</code>	Wallet-like table: capacity sources, states, expiry and today's observed usage.
<code>quotamesh tail</code>	Recent request/fallback feed.
<code>quotamesh doctor [key]</code>	Explicit provider/model test.
<code>quotamesh add</code>	Add a credential.
<code>quotamesh import-env</code>	Import known provider keys already present in process environment.
<code>quotamesh profile test <slug></code>	Dry-run candidate decisions with skip reasons.
<code>quotamesh env <slug></code>	Print integration environment variables.
<code>quotamesh demo</code>	Start scripted fake providers and show failover safely.
<code>quotamesh reset <key></code>	Clear manual health/quota state after edit/top-up.

A full-screen TUI is deferred. The closest local competitors already have strong TUI surfaces, so duplicating that is not a strategic MVP win.

17. Testing strategy

Most risk is in deterministic selection, failure classification and stream commitment. Test those before polishing UI.

17.1 Fake upstream

A small Starlette/FastAPI test server whose behavior depends on the supplied fake key:

```
fake-200
fake-401
fake-400-badkey
fake-403-model
fake-429-short
fake-429-daily
fake-429-unknown
fake-5xx
fake-slow-first-event
fake-sse-error-first
fake-midstream-cut
fake-context-too-long
```

The same fake upstream powers `quotamesh demo`.

17.2 Unit tests

- Provider classifiers from captured fixtures.
- Cooldown ladder and reset calculations.
- Candidate filtering.
- Same quota-group propagation.
- Paid/trial filters.
- Paid cap checks including unknown price.
- Expiry.
- Raw-body preservation except model/known quirks.
- Redaction/fingerprint logic.
- Dry-run/live candidate parity.

17.3 Integration matrix

Test	Expected
Primary 200	One attempt; no fallback.
Key A 429, Key B 200	A cools for that quota-group/model; B serves.

Test	Expected
Same quota group exhausted	Sibling secret is skipped.
Gemini-style 400 invalid key	Credential invalid; backup serves.
Generic malformed 400	Returned as-is; no fallback.
Context-length error	Next configured target may be tried; no key cooldown.
429 Retry-After 120	No waiting; next candidate.
First SSE event is error	Fall back before commit.
Failure after first valid SSE event	No fallback; stream ends with error.
20 parallel calls; first 429 arrives	No newly selected request goes to cooled key after state update.
allow_paid=false	Paid target is never attempted.
Cap reached	Paid target is skipped.
Unknown price + cap set	Paid target skipped unless explicit override.
Restart in cooldown	Persisted quota cooldown survives.
Wrong Host/foreign Origin	Local management request rejected.
Redirect from custom endpoint	Not followed; secret not forwarded to redirected host.
Log/database write failure	Upstream response still succeeds; diagnostic counter increments.
Dry run	Same first eligible candidate as live request under same state snapshot.

17.4 Release acceptance

- Fresh install to first proxied request in under five minutes.
- Works with only two keys and one provider.
- Wallet shows free/trial/paid capacity with truth labels.
- User can create a project profile and copy integration.
- Free-only project can never touch a paid key.
- Fallback occurs on a retryable pre-commit failure.
- No provider switch after stream commitment.
- Trial expiry blocks an expired credential.
- Usage outside QuotaMesh is explicitly described as invisible unless provider data exists.
- No prompt/response bodies stored.
- No full API key appears in logs, activity or management responses.

- quotamesh demo visibly demonstrates failure -> fallback -> paid guard.

18. Build plan for one developer

Build the engine and fake provider first. The UI should be a thin view over proven state, not the place where business logic is invented.

Phase	Rough effort	Output	Done when
0. Foundation	1-2 days	Package, config, schema, registry loader, fake upstream, architecture rules in repo	Tests run; fake upstream starts.
1. Pass-through	2-3 days	Local auth, generic upstream transport, one provider, stream/non-stream, metadata logging	OpenAI SDK works through localhost.
2. Decision engine	3-4 days	Body-aware classify, candidates, key pools, quota groups, cooldowns, caps, expiry, persistence	Full engine integration table passes.
3. Wallet + Profiles	3-4 days	Capacity Wallet, Keys, Project Profiles, daily rollups, history	Mixed free/trial/paid state is understandable without reading logs.
4. Explain + Connect	2-3 days	Available Now, Explain Route, integration snippets, Doctor, demo	User can understand and connect a project in minutes.
5. Ship	2-3 days	Access Catalog, docs, PyPI/uvx, screenshots/GIF, clean-machine test	<code>uvx quotamesh start</code> works on Windows/macOS/Linux target set.

Total rough focused effort: 13-19 developer days with strong AI coding assistance. This is an estimate, not a commitment. Provider quirks and packaging are the most likely sources of variance.

18.1 Implementation priority inside each phase

1. Table tests first.
2. Pure function second.
3. Integration shell third.
4. UI last.

For every provider error encountered in real use, save a sanitized fixture. Over time those fixtures become more valuable than handwritten provider assumptions.

18.2 Scope guard

If development begins to require any of these before the release acceptance list passes, stop and cut:

```
Redis
PostgreSQL
background workers
OAuth account linking
desktop shell
model ranking
AI router
MCP / agents
Responses translation
Messages translation
20+ provider-specific modules
```

19. Risks and deliberate trade-offs

Risk / trade-off	Likelihood	MVP response
Competitors already cover key rotation/capacity	High	Differentiate around mixed-capacity ownership, project allocation and spend safety; do not claim primitive uniqueness.
Provider limits/model names change	High	Do not route from hardcoded limit tables; use live evidence and dated registry metadata.
Local usage undercounts direct usage	High	Permanent “through QuotaMesh only” label.
Cross-provider models are not equivalent	High	Only use fallback targets user explicitly configures; no hidden quality equivalence.
Compatibility layers drift	Medium	Raw pass-through, minimal quirks, fixture tests.
Plaintext local secrets	Medium	Accept for MVP with cheap hardening and environment references; keyring later.
Paid cap overshoot	Medium	Local caps checked before selection; warn that in-flight cost can exceed cap slightly.
Account/terms action	Provider-dependent	Neutral product language and terms notes; never automate account multiplication.
Access Catalog becomes stale	Medium	Links + last_verified only; avoid numeric free-quota claims.
Scope creep	High	Maintain explicit reject/defer list and phase gate.

20. Roadmap after MVP proves useful

v0.2 - Local polish and secret handling

- OS keyring/native credential storage.
- `.env` file import with preview and explicit confirmation.
- Simple model capability metadata for Available Now filters.
- Rotate strategy.
- Richer trend views.
- Native Anthropic provider adapter if demand justifies it.
- Registry update command.

v0.3 - Protocol compatibility

- `/v1/responses` translation for Codex-class clients.
- `/v1/messages` translation for Claude Code/native Anthropic clients.
- Additional compatibility tests.

v0.4 - Broader inference surfaces

- Embeddings.
- Additional endpoint pass-through where correctness is clear.
- Optional capability-aware validation.

Later only if users ask

- Local models as normal providers.
- Per-project local virtual keys.
- Sync/team features.
- Deterministic latency/cost strategies.
- Hosted mode.

Smart/semantic routing should remain optional and late. It is not the reason QuotaMesh deserves to exist.

21. Architecture decision register

ADR	Decision	Status
ADR-001	Local web app, not native shell	ACCEPTED
ADR-002	One FastAPI/Uvicorn process	ACCEPTED
ADR-003	SQLite local persistence	ACCEPTED
ADR-004	Plaintext local secrets for MVP behind repository abstraction	ACCEPTED WITH DEFERRED HARDENING
ADR-005	OpenAI Chat Completions first	ACCEPTED
ADR-006	Raw JSON pass-through; no full upstream schema model	ACCEPTED
ADR-007	Product concept is Capacity Wallet + Project Profiles	ACCEPTED
ADR-008	Project Profile is the route; no separate project subsystem	ACCEPTED
ADR-009	Explicit saved target order; no AI model router	ACCEPTED
ADR-010	Key pools per target	ACCEPTED
ADR-011	Persist quota state per (quota group, model)	ACCEPTED
ADR-012	Unknown 429 -> cooldown ladder, not fake exhaustion	ACCEPTED
ADR-013	Paid disabled by default; caps are local guardrails	ACCEPTED
ADR-014	Provider/local/manual/unknown truth source	ACCEPTED
ADR-015	No provider switch after stream commit	ACCEPTED
ADR-016	Generic transport + registry + small classifier hooks	ACCEPTED
ADR-017	First-class provider count stays deliberately small	ACCEPTED
ADR-018	No background quota/health polling	ACCEPTED
ADR-019	Access Catalog is onboarding data, not routing truth	ACCEPTED
ADR-020	No prompt/response body storage or telemetry by default	ACCEPTED

ADR	Decision	Status
ADR-021	Host/Origin checks + random local key in MVP	ACCEPTED
ADR-022	Responses/Messages protocol translation deferred	ACCEPTED
ADR-023	No Postgres/Redis/Docker requirement	ACCEPTED
ADR-024	No semantic routing, agents, MCP, prompt management or caching	ACCEPTED

22. Recommended README / launch story

22.1 Tagline

Your local AI API capacity wallet.

22.2 One-line description

Connect the free, trial and paid AI API access you already own, allocate it to local project profiles, and keep projects running with explainable failover and hard paid-use guardrails.

22.3 Demo story

A 20-30 second recording should show:

1. Capacity Wallet with two free keys and one paid key.
2. `voice-agent` profile with paid fallback off.
3. "Available Now" showing one free key cooling and another eligible.
4. Send request; first source gets scripted 429; second succeeds.
5. Toggle simulated all-free failure; paid key is shown **BLOCKED**.
6. Enable paid backup with a \$1/day cap; dry-run now shows paid candidate eligible.

This tells the product story more clearly than a provider-count screenshot.

22.4 Claims to avoid

Never publish:

- unlimited free AI;
- multiply your quotas;
- bypass limits;
- 100% accurate remaining quota;
- works with every OpenAI-compatible feature;
- cheapest/fastest automatic model;

- supports Claude Code/Codex natively before those protocol endpoints exist.

23. Source reconciliation summary

From the v0.1 PDF [F1]

Kept:

- local-first one-process architecture;
- FastAPI/httpx/SQLite;
- OpenAI-compatible local endpoint;
- explicit routes and failover;
- paid opt-in;
- provider/local/manual/unknown truth model;
- passive health + Doctor;
- no enterprise/distributed scope.

Replaced or corrected by v0.2/final spec:

- per-key health as the main quota grain;
- status-only failure classification;
- full request Pydantic modelling;
- many provider adapter classes;
- fixed/optional local placeholder auth;
- route policies that could silently reorder user targets;
- pruning request logs without durable rollups.

From the v0.2 Markdown [F2]

Kept as the technical backbone:

- `(quota group, model)` state;
- body-aware classifier;
- raw JSON pass-through;
- data-driven providers;
- target key pools;
- dry-run visibility;
- local security baseline;
- usage rollups;
- fake upstream test strategy;
- explicit protocol compatibility limits.

Simplified in this final spec:

- Project Profile replaces Route in user-facing language.
- `private` automated data-use routing is deferred.
- provider breadth is reduced to tested core + presets.
- price updater is deferred.
- unknown 429 is not escalated to daily exhaustion without evidence.
- some short target states stay in memory instead of adding schema.

From the pasted feature proposal [F3]

Promoted:

- Capacity Wallet as the home mental model.
- Project Profiles.
- paid/trial guardrails.
- Available Now.
- integration snippets.
- Explain Route.
- trial expiry and local credit accounting.
- Access Catalog.

Narrowed after current competitor research:

- Capacity status, health race and free API discovery are useful but **not** unique enough to be headline differentiation because freellmpool now ships similar capacity/health/discovery tooling. [W3]
- The real differentiated composition is **mixed BYOK capacity + per-project allocation + explicit paid safety + explanation.**

24. Web research references

Checked 2 October 2026. Provider behavior and competitor features can change; re-verify before release claims.

[W1] LiteLLM Router - Load Balancing

<https://docs.litellm.ai/docs/routing>

Broad routing, load balancing, cooldowns, fallbacks and Redis-backed production state.

[W2] Mirrowel LLM-API-Key-Proxy

<https://github.com/Mirrowel/LLM-API-Key-Proxy>

Local universal gateway with key rotation, model-aware cooldowns, credential management, TUI and multi-protocol compatibility.

[W3] freellmpool 0.11.x / capacity tooling

<https://pypi.org/project/freellmpool/0.11.4/>

<https://github.com/0xsr/freellmpool>

Free-tier pooling plus capacity status, provider health, provider/key discovery and dashboard.

[W4] Free LLM Gateway

<https://github.com/MrFadiAi/free-llm-gateway>

24+ providers, 260+ free models, fallback and smart routing claims.

[W5] ModelMesh Lite

<https://pypi.org/project/modelmesh-lite/>

Capability-based routing, free-tier aggregation and multiple strategies.

[W6] Vercel AI Gateway

<https://vercel.com/ai-gateway>

Hosted gateway with broad model/provider support, routing, billing and policy controls.

[W7] Vercel AI Gateway provider sorting

<https://vercel.com/changelog/sort-providers-by-cost-latency-or-throughput-on-ai-gateway>

Dynamic cost, TTFT and throughput sorting.

[W8] Vercel AI Gateway budgets

<https://vercel.com/docs/ai-gateway/observability-and-spend/budgets>

Budgets at team/project/API-key/user scopes.

[W9] Portkey AI Gateway

<https://portkey.ai/features/ai-gateway>

Fallbacks, retries, load balancing, observability and production gateway features.

[W10] Google Gemini API rate limits

<https://ai.google.dev/gemini-api/docs/rate-limits>

Project-scoped limits, model-specific limits and Pacific-midnight daily reset behavior.

[W11] OpenAI API rate limits

<https://developers.openai.com/api/docs/guides/rate-limits>

Rate-limit headers including remaining requests/tokens and reset metadata.

[W12] Anthropic OpenAI SDK compatibility

<https://platform.claude.com/docs/en/cli-sdks-libraries/libraries/openai-sdk>

Compatibility layer positioning and limitations.

[W13] Gemini OpenAI compatibility

<https://ai.google.dev/gemini-api/docs/openai>

Official OpenAI-library-compatible endpoint and current beta limitation note.

[W14] Groq Projects

<https://console.groq.com/docs/projects>

Project-specific keys/rate limits under organization ceilings; some token limits remain organization-level.

[W15] Groq Acceptable Use & Responsible AI Policy

<https://console.groq.com/docs/legal/ai-policy>

Restriction against exceeding published limits through multiple accounts/organizations.

[W16] OpenCode providers / custom base URL

<https://opencode.ai/docs/providers/>

Custom provider base URL support, useful for one-click QuotaMesh integration templates.

[W17] OpenAI Codex Responses API proxy

<https://github.com/openai/codex/blob/main/codex-rs/responses-api-proxy/README.md>

Current Codex custom proxy example using `/v1/responses` and `wire_api='responses'`.

25. Final build checklist

- ☐ `uvx quotamesh start` launches one local process with no external infrastructure.
- ☐ Random local bearer key and dashboard bootstrap session are created on first run.
- ☐ Capacity Wallet groups FREE / TRIAL / PAID and never fabricates remaining quota.
- ☐ Credential add/test requires only provider + secret + plan in the normal path.
- ☐ Multiple secrets can share a quota group.
- ☐ Project Profile is the single user-facing routing concept.
- ☐ `qm/<profile>` works through `/v1/chat/completions` in stream and non-stream mode.
- ☐ Key pools and `(quota group, model)` cooldowns work.
- ☐ Body-aware error classification handles invalid-key and quota edge cases.
- ☐ Paid credentials are impossible to use unless the profile explicitly allows them.
- ☐ Local paid caps skip candidates when reached.
- ☐ Trial expiry excludes a credential automatically.
- ☐ Available Now and Explain Route use the exact live candidate function.
- ☐ Integration snippets exist for Python, Node, cURL, env and OpenCode.
- ☐ History explains every fallback/skip without storing prompts or responses.
- ☐

- Doctor is manual/on-demand only.
- Access Catalog contains only dated links/notes, not volatile quota promises.
- Unknown 429 stays an honest cooldown unless reset/exhaustion evidence exists.
- No provider switch after the first valid SSE event is committed.
- Daily usage rollups survive request-log pruning.
- Logs and management responses never reveal a full provider key.
- Custom provider redirects are not followed.
- Fake upstream test matrix passes.
- `quotamesh demo` demonstrates the product story without real credentials.
- README does not advertise quota circumvention or unlimited free access.
- README states that local usage excludes calls made outside QuotaMesh.
- README states Chat Completions compatibility honestly; Responses/Messages are later.

If all of the above works, ship. Do not add smart routing, more protocols, more providers or enterprise infrastructure first.